

Kubernetes Cluster Orchestration

Part II – Deployments

Dennis M. Pöpperl

Computer Vision Lab

Winter semester 2022

Content

1 Example Application: Wordpress

2 Manifests

- Introduction
- Persistent Volumes
- Services
- Deployments

3 Executing the Configuration

Example Application: Wordpress

Content

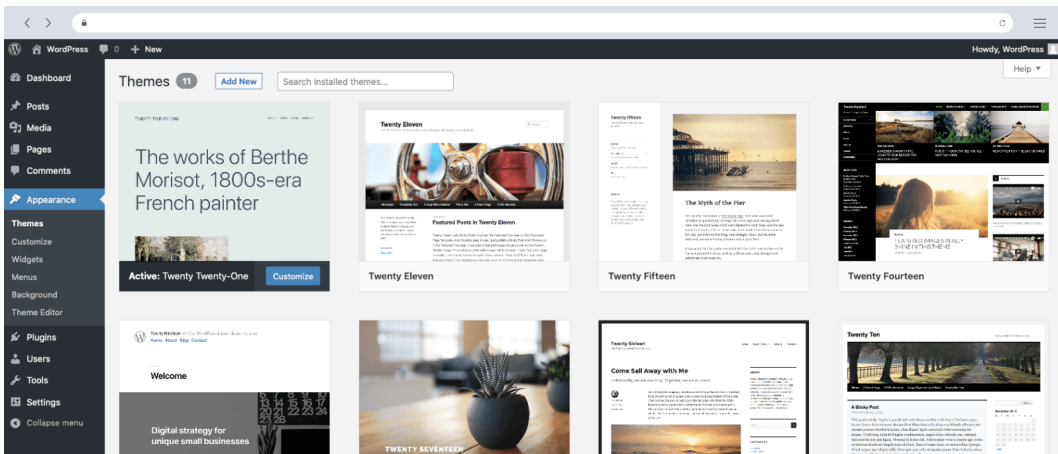
1 Example Application: Wordpress

2 Manifests

- Introduction
- Persistent Volumes
- Services
- Deployments

3 Executing the Configuration

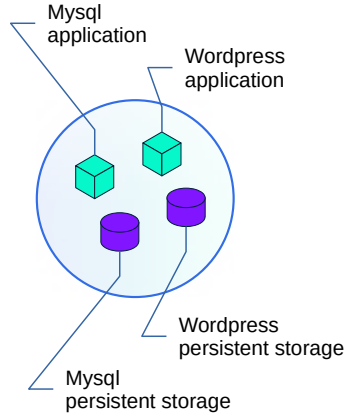
Example Application: Wordpress



Example Application: Wordpress

Wordpress is the most widely used framework for building dynamic websites and blogs.

- User handling, comment sections
- Extensible, themeable
- SQL database backend



Manifests

Content

1 Example Application: Wordpress

2 Manifests

- Introduction
- Persistent Volumes
- Services
- Deployments

3 Executing the Configuration

Manifests – Introduction

Declarative configuration

- Describe what you *need*, not how to get there
- Cluster manager decides necessary steps
- Idempotent

Manifest files

- XML-like tree structure with key-value pairs
- Declare services, Pods deployments etc.

Manifests – Introduction

Top-level declaration

- Manage secrets for your application
- Provide the sources for deployments
- Handed to cluster manager

Listing 1: kustomization.yaml

```
secretGenerator:  
  - name: mysql-pass  
    literals:  
      - password=YOUR_PASSWD  
resources:  
  - mysql.yaml  
  - wordpress.yaml
```


Manifests – Persistent Volumes

- Permanent file storage
- Referenced by name
- Fulfilled by StorageClass config
- Mounted into filesystem tree by container

Caution

Filesystem tree in container is read-only!

Listing 2: mysql.yaml (part)

```
kind: PersistentVolumeClaim
metadata:
  name: mysql-pv-claim
  labels:
    app: wordpress
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi
```

Manifests – Services

- Outside access to deployment
- IP address
 - Part of Kubernetes address space
 - Dynamically assigned upon cluster creation
 - Resolvable by metadata name
- Specifies network port
- Selector chooses app

Listing 3: mysql.yaml (part)

```
kind: Service
metadata:
  name: wordpress-mysql
  labels:
    app: wordpress
spec:
  ports:
    - port: 3306
  selector:
    app: wordpress
    tier: mysql
```

Manifests – Deployments

- Inform selector which labels should use this deployment
- Choice of strategy:
 - Recreate Replace all at once
 - Rolling Replace one-by-one
- Template defines how to instantiate a new pod

Listing 4: mysql.yaml (part)

```
kind: Deployment
metadata:
  name: wordpress-mysql
  labels:
    app: wordpress
spec:
  selector:
    matchLabels:
      app: wordpress
      tier: mysql
  strategy:
    type: Recreate
  template:
```

Manifests – Deployments

Container list

- Image name and tag in registry
- Port exposure
- Volume mounts for external data access

Volumes list

- Instantiates volumes with a provider
- PV claims, ephemeral storage ...

Listing 5: mysql.yaml (part)

```
containers:
  - image: mysql:5.6
    ports:
      - containerPort: 3306
    volumeMounts:
      - name: db-persist
        mountPath: /var/...
    env:
volumes:
  - name: db-persist
    persistentVolumeClaim:
      claimName: db-pv
```

Executing the Configuration

Content

1 Example Application: Wordpress

2 Manifests

- Introduction
- Persistent Volumes
- Services
- Deployments

3 Executing the Configuration

Executing the Configuration

Apply the configuration and list created services:

```
kubectl apply -k ./  
kubectl get services wordpress
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
wordpress	LoadBalancer	10.0.0.89	<pending>	80:32406/TCP

- Internal HTTP-port mapped to random cluster port
- Service is ready to be reverse proxied

Executing the Configuration

List created Pods:

```
kubectl get pods
```

NAME	READY	STATUS	RESTARTS
wordpress-mysql-1894417608-x5dzt	1/1	Running	0

- Pods are created as needed
- Can run concurrently

Executing the Configuration

List persistent volume claims:

```
kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES
mysql-pv-claim	Bound	pvc-8cbd7b2e	20Gi	RWO
wp-pv-claim	Bound	pvc-8cd0df54	20Gi	RWO

- If no matching volume exists, a new one is dynamically created
- Claims and matching volume bound to Pod here

Appendix

Most images and content taken from:

<https://kubernetes.io/docs/tutorials/stateful-application/mysql-wordpress-persistent-volume>